Verified Search and Hardware Interfaces

Example Author

Example Institution

Course / Programming Assignment

2026-11-11



$$\dot{x} = \sigma(y - x),$$
$$\dot{y} = x(\rho - z) - y,$$
$$\dot{z} = xy - \beta z$$

σ=10, ρ=28, β=$\frac{8}{3}$

1 Specification and interface

`binary_search(values, target)` accepts an ascending sorted sequence and returns the first matching zero-based index, or -1 if absent. The caller owns sorting; the function does not mutate its input. This is an algorithm example, not an externally measured benchmark.

Table 1: Interface contract.

Field	Contract
Input	Sorted indexable sequence and comparable target
Output	First matching index; -1 when absent
Precondition	Elements support ordering against the target
Failure mode	Unsorted inputs violate the contract

2 Algorithm and complexity

Algorithm 1 Lower-bound search

1: $\ell \leftarrow 0, h \leftarrow n$
2: **while** $\ell < h$ **do**
3: $\quad m \leftarrow \lfloor (\ell + h)/2 \rfloor$
4: $\quad$ **if** $a_m < x$ **then**
5: $\quad\quad \ell \leftarrow m + 1$
6: $\quad$ **else**
7: $\quad\quad h \leftarrow m$
8: $\quad$ **end if**
9: **end while**
10: **return** ℓ if $\ell < n$ and $a_\ell = x$; otherwise -1

The candidate interval shrinks geometrically, so the search requires $O(\log n)$ comparisons and $O(1)$ auxiliary space. The invariant is that the lower-bound position remains in $[\ell, h]$.

3 Implementation and architecture

```python
def binary_search(values, target):
    """Return first matching index, or
    -1. Input must be sorted."""
    lo, hi = 0, len(values)
    while lo < hi:
        mid = (lo + hi) // 2
        if values[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo if lo < len(values) and
    values[lo] == target else -1
```

Listing 1: Implementation of the stated search contract.

Architecture and language examples

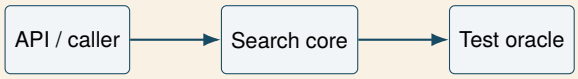


Figure 1: Documentation view of the component boundary.

```rust
fn main() {
    let cycles: u64 = 42;
    println!("cycles = {}", cycles);
}
```

Listing 2: Rust highlighting example; the build only typesets this file.

```asm
# Illustrative arithmetic, not a complete
    executable.
addi t0, zero, 5
addi t1, zero, 7
add  a0, t0, t1
```

Listing 3: RISC-V assembly highlighting example.

4 Test cases, debugging, and benchmarking

For a bug report, record the minimal failing input, expected/actual behavior, root cause, and regression check. For benchmarks, record input size, repetitions, compiler flags, hardware, timing method, and

Table 2: Expected search results; execute the supplied Python test script to verify.

Input	Target	Expected
`[]`	4	−1
`[2, 4, 6]`	4	1
`[2, 4, 6]`	5	−1
`[2, 2, 2]`	2	0

uncertainty. No runtime numbers are supplied by this template.